Simulator Lab Sheet 2: Building a Five-Stage Pipeline

on the RV32I Simulator (`riscv_sim_base`)

1. Objectives

By the end of this lab you will have converted a working single-cycle RV32I simulator into a five-stage pipelined one, *to existing simulator code*. You will:

1. Represent the four pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) as C structures.
2. Separate the work of one instruction across five stages that each advance once per clock.
3. Generate control signals in the decode stage and carry them with the instruction.
4. Measure CPI and cycle count, and explain the numbers you get.

2. What You Are Given

The handout `riscv_sim_base/` is a complete, working **single-cycle** RV32I simulator: an assembler, a byte-addressable memory, an ISA registry (instruction dispatch table), a decoder, and a CLI debugger. Build and run it first:

```
cd riscv_sim_base
make
./rvsim tests/sample1_sum_loop.s --batch
```

You should see the program print `55`, a register dump with `t1 = 0x00000037`, and `status = ECALL_EXIT  cycles = 53`. (**Do not start Task 1 until this works.)**

The single-cycle engine lives in `core/executor.c` and is only 30 lines. Read it now — it is the thing you are about to split into five pieces:

```
CPUStatus execute_cycle(CPUState *cpu, Memory *mem) {
    memory_read_word(mem, cpu->pc, &raw);           /* <- becomes IF  */
    decode_instruction(raw, &instr, &handler);      /* <- becomes ID  */
    cpu->next_pc = cpu->pc + 4;
    handler->execute(cpu, mem, &instr);             /* <- becomes EX + MEM + WB */
    cpu->pc = cpu->next_pc;
    cpu->cycle_count++;
}
```

This single-cycle path stays in the build untouched. It is your **reference oracle**: your pipeline must leave the register file in exactly the same final state.

Rules of engagement

- **Do not modify** anything in `core/`, `isa/`, `assembler/` or `ui/`. Unless you need to absolutely. You may read them freely.
- All your new code goes in a **new directory** `pipeline/`.
- You make exactly two small edits outside it: three lines in `main.c` and two lines in `Makefile` (Task 6). Nothing else.
- You *reuse* `decode_instruction()` from `core/decoder.c` and `cpu_read_reg()` / `cpu_write_reg()` / `memory_*()`. Do not re-implement field extraction or immediate sign-extension — that is already correct.

Files you will create

```
riscv_sim_base/
+-- pipeline/
|   +-- control.h              Task 1     ControlSignals struct + prototype
|   +-- control.c              Task 1     control_decode()
|   +-- pipeline_state.h       Task 2     IFID / IDEX / EXMEM / MEMWB / Pipeline structs
|   +-- pipeline.c             Tasks 3  the five stages, flushes
+-- tests/sample3_hazards.s    (provided with this sheet - copy it into tests/)
```

3. Background: One Cycle Becomes Five Stages

In the single-cycle machine, one call to `execute_cycle()` completes one instruction. In the pipelined machine, one call to `pipeline_cycle()` is one *clock tick*, and up to five instructions are in flight at once:

```
cycle ->      1     2     3     4     5     6     7
I1            IF    ID    EX    MEM   WB
I2                  IF    ID    EX    MEM   WB
I3                        IF    ID    EX    MEM   WB
```

The instruction in EX during cycle 3 is *not* the one in IF during cycle 3. What makes this work is that each stage never talks to the next stage directly; it writes into a **pipeline register** (a latch), and the next stage reads that latch on the *following* clock.

The one rule that decides whether your pipeline works

Within one clock tick, every stage must read the **old** latch contents and write the **new** ones. If EX overwrites the ID/EX latch before ID has read it, you have built a time machine, not a pipeline. The simplest way to guarantee this in C is to snapshot the whole latch set at the top of the cycle and commit it at the bottom:

```
CPUStatus pipeline_cycle(Pipeline *p, CPUState *cpu, Memory *mem) {
    Pipeline nx = *p;        /* nx = next state; p = current (old) state */

    /* ... every stage READS from p->..., WRITES into nx.... ... */

    *p = nx;                 /* commit: one clock edge */
    return cpu->status;
}
```

With this idiom the order you write the five stages in does not matter. Use it.

Instruction subset your pipeline must support

- **R-type:** `add sub and or xor sll srl sra slt sltu`
- **I-type:** `addi andi ori xori slti sltiu slli srli srai`
- **U-type:** `lui auipc` · **Loads:** `lb lh lw lbu lhu` · **Stores:** `sb sh sw`
- **Branches:** `beq bne blt bge bltu bgeu` · **Jumps:** `jal jalr` · **System:** `ecall`

That is all of RV32I, which is what the provided test programs use. Nothing else is required.

4. Task 1 — Control Signals

Files: `pipeline/control.h`, `pipeline/control.c`

In hardware, the decode stage inspects the opcode and produces control signals that travel down the pipe with the instruction. Every later stage then acts purely on those signals, without re-examining the opcode. Build exactly that.

In `control.h` declare:

```
typedef enum { WB_NONE = 0, WB_ALU, WB_MEM, WB_PC4 } WbSrc;

typedef struct {
    int   reg_write;    /* does this instruction write rd?         */
    WbSrc wb_src;       /* what value gets written back?           */
    int   mem_read;     /* MEM stage performs a load               */
    int   mem_write;    /* MEM stage performs a store              */
    int   is_branch;    /* conditional branch - EX decides         */
    int   is_jump;      /* jal / jalr - EX always redirects        */
    int   is_system;    /* ecall - handled at WB                   */
    int   uses_rs1;     /* needed by the hazard detector (Task 4)  */
    int   uses_rs2;
} ControlSignals;

ControlSignals control_decode(const DecodedInstr *i, const InstructionHandler *h);
```

In `control.c`, implement `control_decode()` as a single `switch` on `h->format` (the `InstrFormat` enum from `core/isa_registry.h`). Note that you get the format for free from the ISA registry — you do not need to classify opcodes yourself. Fill in the signals according to this table:

format	reg_write	wb_src	mem_read	mem_write	is_branch	is_jump	uses_rs1	uses_rs2
FMT_R	1	WB_ALU	0	0	0	0	1	1
FMT_I, FMT_I_SHAMT	1	WB_ALU	0	0	0	0	1	0
FMT_U	1	WB_ALU	0	0	0	0	0	0
FMT_I_LOAD	1	WB_MEM	1	0	0	0	1	0
FMT_S	0	WB_NONE	0	1	0	0	1	1
FMT_B	0	WB_NONE	0	0	1	0	1	1
FMT_J	1	WB_PC4	0	0	0	1	0	0
FMT_I_JALR	1	WB_PC4	0	0	0	1	1	0
FMT_I_SYS	0	WB_NONE	0	0	0	0	0	0

Checkpoint: `gcc -c -Icore -Iisa -Ipipeline pipeline/control.c` compiles with no warnings under `-Wall -Wextra`.

5. Task 2 — The Pipeline Latches

File: `pipeline/pipeline_state.h`

Define one struct per latch. A latch holds everything the *downstream* stages still need; nothing more. Required fields:

Latch	Fields
IFID	`int valid; uint32_t pc; uint32_t raw; int fault;`
IDEX	`int valid; uint32_t pc; int fault; DecodedInstr instr; const InstructionHandler *handler; ControlSignals ctrl; uint32_t rs1_val, rs2_val;`
EXMEM	as IDEX minus `rs1_val/rs2_val`, plus `uint32_t alu_result; uint32_t store_val; uint32_t fault_addr;`
MEMWB	as EXMEM but with `uint32_t wb_val;` in place of `alu_result/store_val`

Then define the pipeline itself and its statistics counters:

```
typedef struct {
    IFID if_id;  IDEX id_ex;  EXMEM ex_mem;  MEMWB mem_wb;
    uint64_t cycles, retired, stalls, flushes;
} Pipeline;

void      pipeline_init (Pipeline *p);   /* memset the whole thing to zero */
CPUStatus pipeline_cycle(Pipeline *p, CPUState *cpu, Memory *mem);
void      pipeline_run_batch(Pipeline *p, CPUState *cpu, Memory *mem, uint64_t max_cycles);
```

Two fields deserve explanation:

- `valid` — a latch can be empty. It is empty at startup, and it is empty whenever you inject a **bubble** (a stall) or kill an instruction (a flush). A stage with `valid == 0` must do nothing at all this cycle. Every stage therefore begins with `if (!p->xx_yy.valid) { /* no work */ }`.
- `fault` — define `PF_NONE 0`, `PF_ILLEGAL 1`, `PF_FETCH 2`, `PF_MEM 3`. A pipelined machine may *not* halt the moment something goes wrong, because the instructions ahead of the faulting one have not finished yet. So a fault is recorded in the latch and carried down the pipe; only the WB stage turns it into a `CPUStatus`. This is called a **precise exception**, and you need it for a concrete reason: after the final `ecall` your IF stage keeps fetching into the `.data` section and will decode `0x00000000` as illegal. If you halt in ID, the program dies before its `ecall` ever reaches WB.

6. Task 3 — The Five Stages

File: `pipeline/pipeline.c`

Write `pipeline_cycle()` using the snapshot/commit shell from §3. Implement each stage as described below. Start with the `nx.xx_yy.valid = 0` default for each latch you are about to overwrite, so that a stage with nothing to do naturally produces a bubble.

WB — write back (reads `p->mem_wb`)

1. If `fault != PF_NONE`: set `cpu->status` (`CPU_ILLEGAL_INSTRUCTION` or `CPU_MEM_FAULT`), fill `cpu->fault_addr` and `cpu->fault_msg`, and return — the machine stops here. This is the only place a fault becomes visible.
2. If `ctrl.reg_write`: `cpu_write_reg(cpu, instr.rd, wb_val)`. (`cpu_write_reg` already discards writes to `x0` — do not re-check.)
3. If `ctrl.is_system`: this is `ecall`. Because the pipeline retires instructions in order, every earlier instruction has already written back, so the register file is exactly correct here — and nowhere earlier. Call the registry's own handler: temporarily set `cpu->pc` to this instruction's `pc`, call `handler->execute(cpu, mem, &instr)`, then restore `cpu->pc`.
4. Increment `retired`.

MEM — memory access (reads `p->ex_mem`, writes `nx.mem_wb`)

Copy `pc`, `fault`, `instr`, `handler`, `ctrl` through, and set `wb_val = alu_result` as the default. Then:

- If `ctrl.mem_read`: `alu_result` is the address. Switch on `instr.funct3` to pick `memory_read_byte/half/word`: `0x0 = lb` (sign-extend), `0x1 = lh` (sign-extend), `0x2 = lw`, `0x4 = lbu` (zero-extend), `0x5 = lhu` (zero-extend). Put the result in `wb_val`.
- If `ctrl.mem_write`: write `store_val` at `alu_result`, sized by `funct3` (`0x0 = sb`, `0x1 = sh`, `0x2 = sw`).
- If a `memory_*` call does not return `MEM_OK`, set `fault = PF_MEM` and `fault_addr` in the outgoing latch. Do not halt.

EX — execute (reads `p->id_ex`, writes `nx.ex_mem`)

Copy the instruction through and set `store_val = rs2_val` (the value a store will write). Then exactly one of three things happens:

- `ctrl.is_branch`: compare `rs1_val` against `rs2_val` according to `funct3` (`0x0 beq`, `0x1 bne`, `0x4 blt`, `0x5 bge`, `0x6 bltu`, `0x7 bgeu`; for `blt`/`bge` cast to `int32_t`). If taken, raise a local `flush` flag and set `redirect_pc = pc + imm` (Task 5). A branch writes no register.
- `ctrl.is_jump`: `alu_result = pc + 4` (the link value that WB will write to `rd`), raise `flush`, and set `redirect_pc` to `pc + imm` for `jal` (opcode `0x6F`) or to `(rs1_val + imm) & ~1u` for `jalr` (opcode `0x67`).
- otherwise: `alu_result = alu(...)`, using a helper you write with the signature `static uint32_t alu(const DecodedInstr *i, uint32_t pc, uint32_t a, uint32_t b)`. It switches on `i->opcode`, then `i->funct3` (and `i->funct7` to tell `add`/`sub` and `srl`/`sra` apart). Three cases are special: `lui` (opcode `0x37`) returns `imm`; `auipc` (`0x17`) returns `pc + imm`; loads and stores return `a + imm`, i.e. the effective address. Mask every shift amount with `& 0x1F`.

ID — decode and read registers (reads `p->if_id`, writes `nx.id_ex`)

1. Call `decode_instruction(if_id.raw, &d, &h)`. If it returns `DECODE_UNKNOWN_INSTRUCTION`, build the outgoing latch with `fault = PF_ILLEGAL` and stop processing it — do not dereference `h`. If the incoming latch already carries `PF_FETCH`, propagate that instead.
2. Call `control_decode()` to get the signals.
3. Run the hazard check (Task 4). If it fires, stall instead of advancing.
4. Otherwise read the register file into the latch: `rs1_val = cpu_read_reg(cpu, d.rs1)`, same for `rs2`. Read *both*, unconditionally — it is harmless, and matches the hardware.

IF — fetch (writes `nx.if_id`)

Fetch from `redirect_pc` if a flush is in progress this cycle, otherwise from `cpu->pc`. Use `memory_read_word()`; if it fails, emit a latch with `fault = PF_FETCH` rather than halting. Then advance the fetch pointer: `cpu->pc = fetch_pc + 4`. Note the pipelined machine uses `cpu->pc` as its *fetch* PC, so at the end of a run it points a few instructions past the last one retired — that is expected and correct.

7. Verification

Step 1: the register files must match

The single-cycle engine is the specification. For each test program, run both engines and diff the register dumps:

```
./rvsim tests/sample1_sum_loop.s --batch    > ref.txt
./rvsim tests/sample1_sum_loop.s --pipeline > pipe.txt
diff <(grep '^x' ref.txt) <(grep '^x' pipe.txt) && echo MATCH
```

Diff only the `x...` register lines: the final `pc` legitimately differs (the pipeline's fetch PC runs ahead) and the cycle counts differ by design.

Step 2: the numbers must be explainable

Copy `sample3_hazards.s` (Appendix A) into `tests/` and run all three programs. Reference figures for the policy specified in Tasks 4 and 5 — naive 3-latch stalling, branches resolved in EX:

Program	single-cycle	pipeline	retired	stalls	flushes	CPI
`sample1_sum_loop.s`	53	108	53	40	11	2.038
`sample2_mem_and_branches.s`	13	26	13	8	1	2.000
`sample3_hazards.s`	18	44	18	19	3	2.444

retired must match the single-cycle cycle count exactly — it is the number of instructions the program executes, and no pipeline may change it. **stalls** and **flushes** may differ slightly from the table if you made a defensible design choice; if they do, say which choice and why in your report. If `retired` is wrong, or the registers differ, you have a bug — not a design choice.

Your counters must satisfy this identity exactly, for every program:

$$\texttt{cycles} = \texttt{retired} + \texttt{stalls} + \texttt{flushes} + 4$$

Every clock tick either retires an instruction, is lost to a stall, is lost to a flush, or is one of the four fill cycles before the first instruction reaches WB. Check it on all three rows of the table above. If it does not hold, one of your counters is being incremented in the wrong place — which is usually the same bug that is corrupting your register file. (You can also verify the flush count by inspection: `sample1` executes `j loop` ten times and takes the `beq` once, which is exactly the 11 flushes shown.)

Step 3: cycle-by-cycle debugging

When a program produces wrong registers, printing the final state tells you nothing. Add a temporary trace at the end of `pipeline_cycle()`:

```
fprintf(stderr, "c%-4llu IF:%08x ID:%c EX:%c MEM:%c WB:%c %s%s\n",
        cycles, if_id.pc, ID?'*':'.', EX?'*':'.', MEM?'*':'.', WB?'*':'.',
        stall ? "STALL " : "", flush ? "FLUSH" : "");
```

Run it on a five-instruction program and check by hand that the diagonal in §3 appears, that a stall repeats a PC in ID, and that a flush is followed by a fetch at the branch target.

Common bugs

Symptom	Likely cause
One instruction silently skipped after every stall	IF was not frozen during the stall (Task 4, step 3)
Registers right, `retired` too high	bubbles counted as retired instructions; only count `valid` instructions leaving WB
Pipeline hangs, `stalls` grows forever	hazard check includes an invalid latch, or compares `rd` when `reg_write` is 0
Everything off by one instruction	a stage read `nx` instead of `p`
`ILLEGAL_INSTRUCTION` just before the program would exit	fault raised in ID instead of carried to WB (Task 2)
Store writes the wrong value	`store_val` taken from the register file in MEM instead of `rs2_val` captured in ID
`jal` leaves `ra` unset	link value `pc+4` not routed through `alu_result` / `WB_PC4`

Appendix: Performance Comparison: Single-Cycle vs Pipelined

The objective of this experiment is to compare the execution of the same program using the original single-cycle simulator and the newly implemented five-stage pipelined simulator.

Students will observe how pipelining overlaps instruction execution and compare the execution cycle counts of both implementations.

Procedure

1. Run the single-cycle simulator.
    - Use the original execution mode to run the supplied sample program.
    - Record the program output, final register values, number of executed instructions, and total execution cycles.
2. Run the pipelined simulator.
    - Run the same program using the five-stage pipeline implemented during this lab.
    - Record the same measurements.
3. Verify the results.
    - Compare the final register values and program output from both executions.
    - The results should match for programs whose instructions and execution behaviour are supported by the current pipeline implementation.
4. Compare execution performance.
    - Calculate the cycles per instruction (CPI) for both implementations.
    - Calculate the percentage change in execution cycle count.
    - Explain the differences observed between the single-cycle and pipelined implementations.
5. Repeat the experiment.
    - Perform the same comparison using another suitable sample program supplied with the simulator.
    - Use a program that the basic pipeline can execute correctly without requiring hazard handling that has not yet been implemented.

Commands

Run the supplied sample program using the original single-cycle execution mode:

```
./rvsim tests/sample1_sum_loop.s --batch
```

Run the same program using the pipelined execution mode:

```
./rvsim tests/sample1_sum_loop.s --pipeline
```

Note: Use the exact pipeline command-line option implemented in your version of the simulator. If the pipeline mode does not yet support this invocation, use the execution method specified in your implementation.

Example: Expected Output — Non-Pipelined vs. Pipelined

1. Input program

Use five independent instructions so there are no data dependencies.

```
addi x1, x0, 10
addi x2, x0, 20
addi x3, x0, 30
addi x4, x0, 40
addi x5, x0, 50
```

2. Expected output: Non-pipelined execution

Assume the non-pipelined implementation executes all five stages sequentially for each instruction.

Output-

Instructions Executed: 5
Total Clock Cycles: 25
CPI: 5.00

Final Register Values:
x1 = 10
x2 = 20
x3 = 30
x4 = 40
x5 = 50

Each instruction takes five clock cycles, so five instructions take $5 \times 5 = 25$ cycles.

3. Expected output: Pipelined execution

Assume a five-stage pipeline, one instruction entering per cycle, and no stalls.

Output-

Instructions Executed: 5
Total Clock Cycles: 9
CPI: 1.80

Final Register Values:
x1 = 10
x2 = 20
x3 = 30
x4 = 40
x5 = 50

The ideal pipeline takes $5 + 4 = 9$ cycles: five instructions plus four additional cycles to fill the pipeline.

4. Performance comparison

Metric	Non-Pipelined	Pipelined
Instructions executed	**5**	**5**
Total clock cycles	**25**	**9**
CPI	**5.00**	**1.80**
Cycle Reduction	**-**	**64%**